

TEMA: Token**Resultados de Aprendizaje:**

-  Que los alumnos puedan agregar seguridad por token a las rutas de la api

DESARROLLO

Autenticación: Es el proceso mediante el cual una API verifica la identidad de un usuario, es decir, comprueba que realmente sea quien dice ser. Normalmente se realiza utilizando un nombre de usuario y contraseña o mediante un token de acceso, como JWT. Una vez autenticado, el sistema puede identificar al usuario que está realizando las solicitudes.

Autorización: Es el proceso mediante el cual la API determina qué acciones puede realizar un usuario que ya ha sido autenticado. Por ejemplo, un administrador puede tener permiso para crear, modificar y eliminar productos, mientras que un vendedor solo puede consultar productos y registrar ventas. Por lo tanto, la autorización controla los permisos y niveles de acceso dentro de la aplicación.

JWT (JSON Web Token) es un formato estándar utilizada para transmitir información de forma segura entre un cliente y un servidor. En una API, se usa comúnmente después de que el usuario inicia sesión correctamente. El servidor genera un token y se lo entrega al cliente; luego, el cliente envía ese token en las siguientes solicitudes para demostrar que ya fue autenticado.

Un JWT contiene información llamada **claims**, por ejemplo el identificador del usuario, su rol o la fecha de expiración del token. Está compuesto por tres partes: **Header**, **Payload** y **Signature**. La firma permite al servidor comprobar que el token no ha sido modificado. Es importante aclarar que, normalmente, el contenido del JWT **no está cifrado**, sino codificado, por lo que no se deben guardar contraseñas ni información sensible dentro de él.

Ejemplo :

Para este ejemplo continuaremos la api comercial que comenzamos en las asiones anteriores y agregaremos un poco mas seguridad a las url , **utilizando OAUT por token**

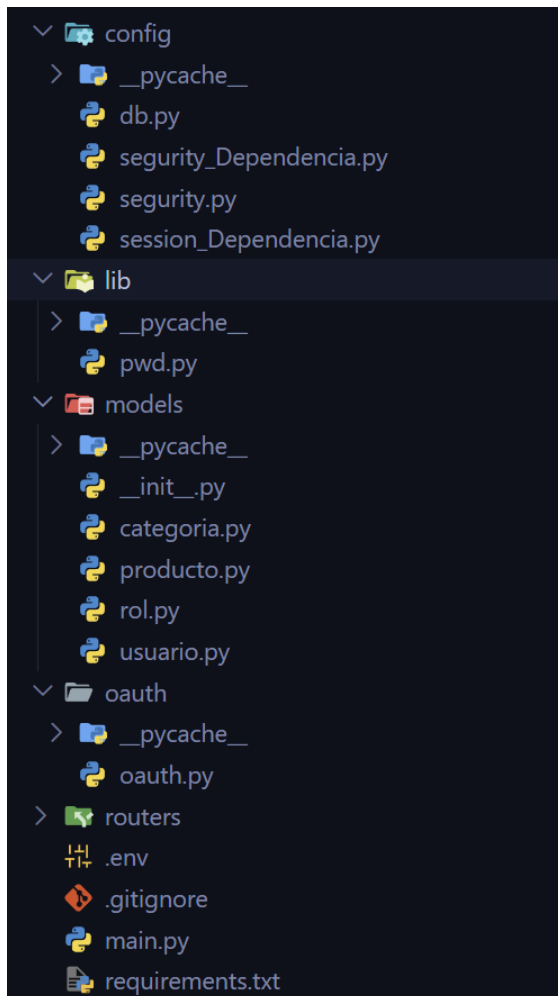
Requermientos

para poder realizar esta practica , debera de contar con:

1. Un gestor de base de datos **mysql**
2. Crear una base de datos llamada **comercial**
3. Haber realizado la guia anteriore de la semana sobre el matenimiento de la tablas

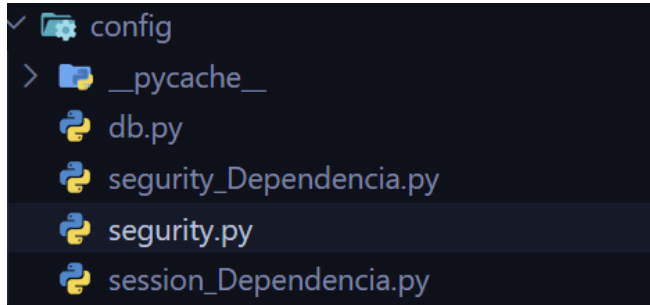
Desarrollo

1. Instale al proyecto las librerías PyJWT con el comando **pip install PyJWT** , esta es una librería que nos permitirá la generación de los token
2. Abra el proyecto creado en la practica anteriores
3. Crearemos los archivos **config/security.py , config/security_Dependencia , oauth/oauth.py** al conjunto de archivos que ya tenemos creados con aterioridad



Seguridad por token

4. Para lograr la seguridad por token crearemos un par de funciones para poder reutilizar el código para ello modificaremos el archivo **config/security.py**



5. Caemos la función para poder crear el token

```
1  from datetime import timedelta, datetime
2  import os
3  from typing import Optional
4
5  from dotenv import load_dotenv
6  from fastapi import Depends, HTTPException, status
7  from fastapi.security import OAuth2PasswordBearer
8  import jwt
9
10 load_dotenv()
11 SECRET_KEY_TOKEN = os.getenv("SECRET_KEY_TOKEN", "123456789")
12 ALGORITHM = "HS256"
13 ACCESS_TOKEN_EXPIRE_MINUTES = 30
14
15 oauth2_scheme = OAuth2PasswordBearer(tokenUrl="oauth/login")
16
17
18 def create_access_token(data: dict, expires_delta: Optional[timedelta] = None):
19     to_encode = data.copy()
20     expire = datetime.utcnow() + (expires_delta or timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES))
21     to_encode.update({"exp": expire})
22     token = jwt.encode(payload=to_encode,
23                         key=SECRET_KEY_TOKEN, algorithm=ALGORITHM)
24     return token
25
```

- **SECRET_KEY_TOKEN** : es la key secreta que debere creaer en sel arachivo **.env** que contiene las varaibles de entorno , de no estar disponible toma los numero 13456789 , no es recomendado dejar esos numero sor defesto , se recomienda una key de 32 caractesre
- **ALGORITHM** : indica el algoritmo utilizado para generar el token
- **ACCESS_TOKEN_EXPIRE_MINUTES** : el timpo en minutos que tendra valides el token , en este caso 30 minutos
- **oauth2_scheme = OAuth2PasswordBearer(tokenUrl="oauth/login")** , muy importante indicar la url donde se realizarea el login o el token , aun no hemos registrtrado dicha url pero lo realizaermos en los siguientes pasos
- la funcion siguenite genera le roken , **token = jwt.encode(payload=to_encode, key=SECRET_KEY_TOKEN, algorithm=ALGORITHM)**

- funcion para decodificar el token, recuede que en los token ocultaremos informacion que pueda ser relevante para recuperar esa informacion utilizaremos esta funcion.

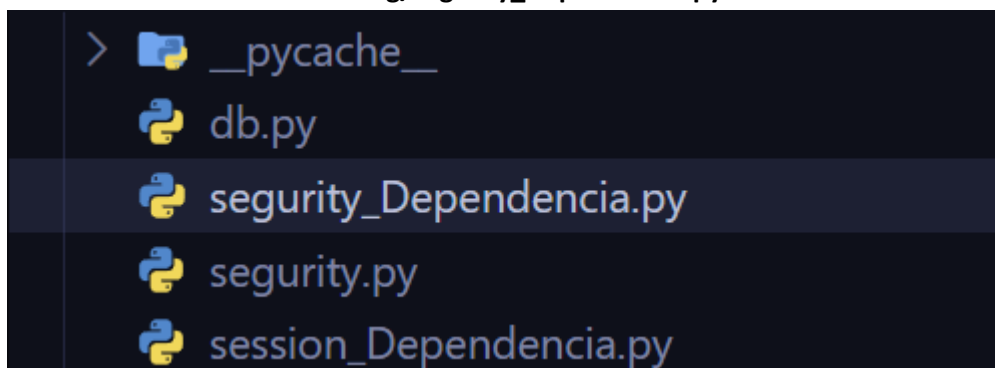
```
def decode_access_token(token: str):  
    payload = jwt.decode(token, key=SECRET_KEY_TOKEN,  
                           algorithms=[ALGORITHM])  
    return payload
```

- ya hemos creado las funciones para crear el token y para decodificarlos, hoy creamos una funcion donde proporcionaremos el token de un usuario y nos retornara la informacion que nosotros agregaremos al token como el id, el username y el id_rol no es recomendado dejar en el token la contraseña

```
3 def get_current_user(token: str = Depends(oauth2_scheme)):  
4     credencial_exception = HTTPException(  
5         status_code=status.HTTP_401_UNAUTHORIZED, detail="No autenticado ", headers={"WWW-Authenticate": "Bearer"})  
6  
7     try:  
8         payload = decode_access_token(token)  
9  
10        id: Optional[int] = payload.get("id")  
11        username: Optional[str] = payload.get("username")  
12        id_rol: Optional[int] = payload.get("id_rol")  
13  
14        if username is None:  
15            raise credencial_exception  
16        return {"id": id, "username": username, "id_rol": id_rol}  
17    except jwt.ExpiredSignatureError:  
18        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,  
19                             detail="Token expirado", headers={"WWW-Authenticate": "Bearer"})  
20    except jwt.InvalidTokenError:  
21        raise credencial_exception  
2
```

Dependencias

- cuando se trabaja desde la api, necesitamos realizar dependencias como las dependencias de la session para poder manipular la base de datos, realizaremos algo similar para poder trabajar con los token
- modificaremos el archivo `config/security_Dependencia.py`



- y agregaremos el código siguiente

```

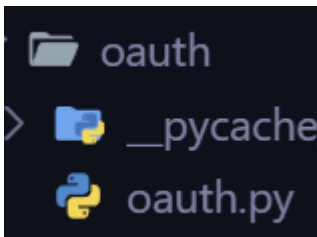
1 from typing import Annotated
2 from fastapi.security import OAuth2PasswordRequestForm
3 from fastapi import Depends
4 from config.security import get_current_user
5
6 OAuth2FormDeDependencia = Annotated[OAuth2PasswordRequestForm, Depends()]
7
8 Token_Dependencia = Annotated[str, Depends(get_current_user)]
9

```

- **OAuth2FormDeDependencia = Annotated[OAuth2PasswordRequestForm, Depends()]**
proporciona de forma predeterminado un esquema de formulario para recepcion de datos de suario donde incluye el username y las password entre algunas cosas mas
- **Token_Dependencia = Annotated[str, Depends(get_current_user)]** , nos permitira pider el token de acceso sobre una url preterminada y por defecto el validara que se tenga dicho token de no terno esta enviara una notificacion de no autorizado

Login de usuario

9. para poder realziar el login y obtenre el token , modificaremos el archivo **oauth/oauth.py**



10. realice el login de la siguiente manera

```
1  from fastapi import APIRouter, status, HTTPException
2  from config.session_Dependencia import SessionDeDependencia
3  from sqlmodel import select
4  from models.usuario import Usuario
5  from lib.pwd import verify_password
6  from config.segurity import create_access_token
7  from config.segurity_Dependencia import OAuth2FormDeDependencia
8
9  router = APIRouter()
10
11
12  @router.post("/oauth/login", status_code=status.HTTP_200_OK)
13  async def login(form_data: OAuth2FormDeDependencia, session: SessionDeDependencia):
14      ... username = form_data.username
15
16      ... consulta = select(Usuario).where(Usuario.username == username)
17      ... usuario = session.exec(consulta).first()
18
19      ... if not usuario:
20      ...     ... raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
21      ...     ... detail="username o password incorrectos")
22      ... # verifica que la contraseña ingresada por el usuario sea correcta
23      ... # comparándola con la contraseña almacenada en la base de datos
24      ... if not verify_password(form_data.password, usuario.password):
25      ...     ... raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
26      ...     ... detail="username o password incorrectos")
27      ... token = create_access_token(
28      ...     ... data={"id": usuario.id, "username": usuario.username, "id_rol": usuario.id_rol})
29      ... #retorna el access token y el tipo de token (bearer) en formato JSON esto es un estandar
30      ... # e seguridad para la autenticacion de usuarios siempre se debe retornar de esta manera
31      ... return {"access_token": token, "token_type": "bearer"}
32
```

- denote que la url , deba ser la misma que ha utilizado cuando configuro el archivo de segurity
- **verify_password** , recuerde que la contraseña en la base de datos ya no es un texto plano por tanto con la fucion verifica si la contaseña ingresada equivale a la almacena en la base de datos

Llama a outh :

11. Hemos creado la ruta para el login pero debemos de agregarla al archivo main.py

```
1  from fastapi import FastAPI, status
2  from contextlib import asynccontextmanager
3  from config.db import crear_db_y_tablas
4  from routers.categoria_router import router as categorias_router
5  from routers.producto_router import router as productos_router
6  from routers.rol_router import router as rol_router
7  from routers.usuario_router import router as usuario_router
8  from oauth.oauth import router as oauth_router
9  import models as models
10
11
12  @asynccontextmanager
13  async def lifespan(app: FastAPI):
14      ... crear_db_y_tablas()
15      ... yield
16
17
18  app = FastAPI(lifespan=lifespan)
19  app.title = "API Tienda la Cachaca"      "Cachaca": Unknown word.
20  app.version = "0.0.1"
21
22
23  @app.get("/", summary="Comprobando estado de api", status_code=status.HTTP_200_OK)
24  async def home():
25      ... return {"message": "ok"}
26
27  # el parámetro tags sirve para agrupar las rutas en la documentation de fastapi
28  app.include_router(categorias_router, tags=["categorias"])
29  app.include_router(productos_router, tags=["productos"])
30  app.include_router(rol_router, tags=["roles"])
31  app.include_router(usuario_router, tags=["usuarios"])
32  app.include_router(oauth_router, tags=["oauth"])
```

12. Hoy es el momento para que usted pueda realizar todas las pruebas correspondientes

13. Si prueba realizar el login desde la documentacion podra notar que esta nos permite agregar el username y las password , esto por que hemos utilizado la dependencia

OAuth2PasswordRequestForm

oauth

POST /oauth/login Login

Parameters

Cancel

Reset

No parameters

Request body ^{required}

application/x-www-form-urlencoded

grant_type
string | (string | null)
pattern: ^password\$
☐ Send empty value

password

username * required
string

cruzovira

password * required
string(\$password)

scope
string

scope

☒ Send empty value

client_id
string | (string | null)

string

☐ Send empty value

client_secret
string | (string | null)(\$password)

☐ Send empty value

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/oauth/login' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/x-www-form-urlencoded' \
  -d 'grant_type=password&username=cruzovira&password=123456&scope=&client_id=string&client_secret=*****'
```

Request URL

http://127.0.0.1:8000/oauth/login

Server response

Code Details

200

Response body

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZiI6Im91dXNlcn5hbWUiOiJjc2V6cm92aCh1IiwiaWF0Ijpmcm9sIjoxIjE4MTA0jE3ODYyOTg5Mjdh9.bfctctVxUpGb-7od4YDgg1b-Tkigpc9H0dW4B81vT0",
  "token_type": "bearer"
}
```



Download

Response headers

```
content-length: 282
content-type: application/json
date: Sun, 09 Aug 2026 17:38:46 GMT
server: unicorn
```

Responses

Code Description

Links

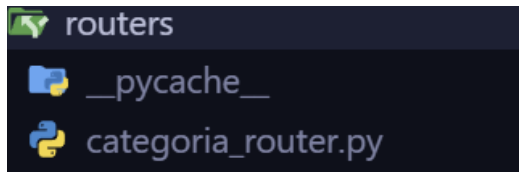
Protegiendo url

para que ya podamos realizar el login hay dos maneras para poder proteger las url

- Que el usuario ya este logueado y posea el token
- O que posea el token pero que además tengo un rol en específico

Protección por token

14. Para poder proteger las url solo por el token es lo más fácil que podemos hacer para ello protegeremos la url de categorías, modificando el archivo `routers/categoria.route.py`

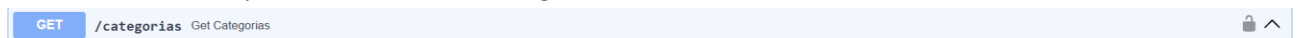


15. Se la siguiente manera

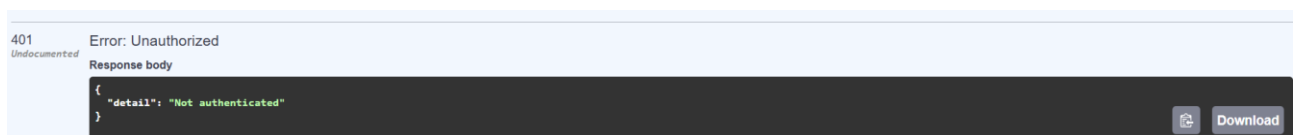
```
1 from datetime import datetime
2 from fastapi import APIRouter, HTTPException, status, Query
3 from models.categoria import Categoria, CategoriaCreate, CategoriaUpdate
4 from sqlmodel import select
5 from config.session_Dependencia import SessionDeDependencia
6 from config.segurity_Dependencia import Token_Dependencia
7
8 router = APIRouter()
9
10
11 @router.get("/categorias", response_model=list[Categoria], status_code=status.HTTP_200_OK)
12 async def get_categorias(session: SessionDeDependencia,
13                          token: Token_Dependencia,
14                          offset: int = Query(0, ge=0),
15                          limit: int = Query(20, ge=1)):
16
17     ...# Obtener los datos del token decodificado
18     ...consulta = select(Categoria).offset(offset).limit(
19     ...limit) ...# Limitar a 'limit' resultados por defecto
20     ...resultado_de_consulta = session.exec(consulta)
21     ...return resultado_de_consulta.all()
22     ...# return session.exec(select(Categoria)).all()
```

- `token: Token_Dependencia` como es una dependencia del token el lo necesita, del usuario

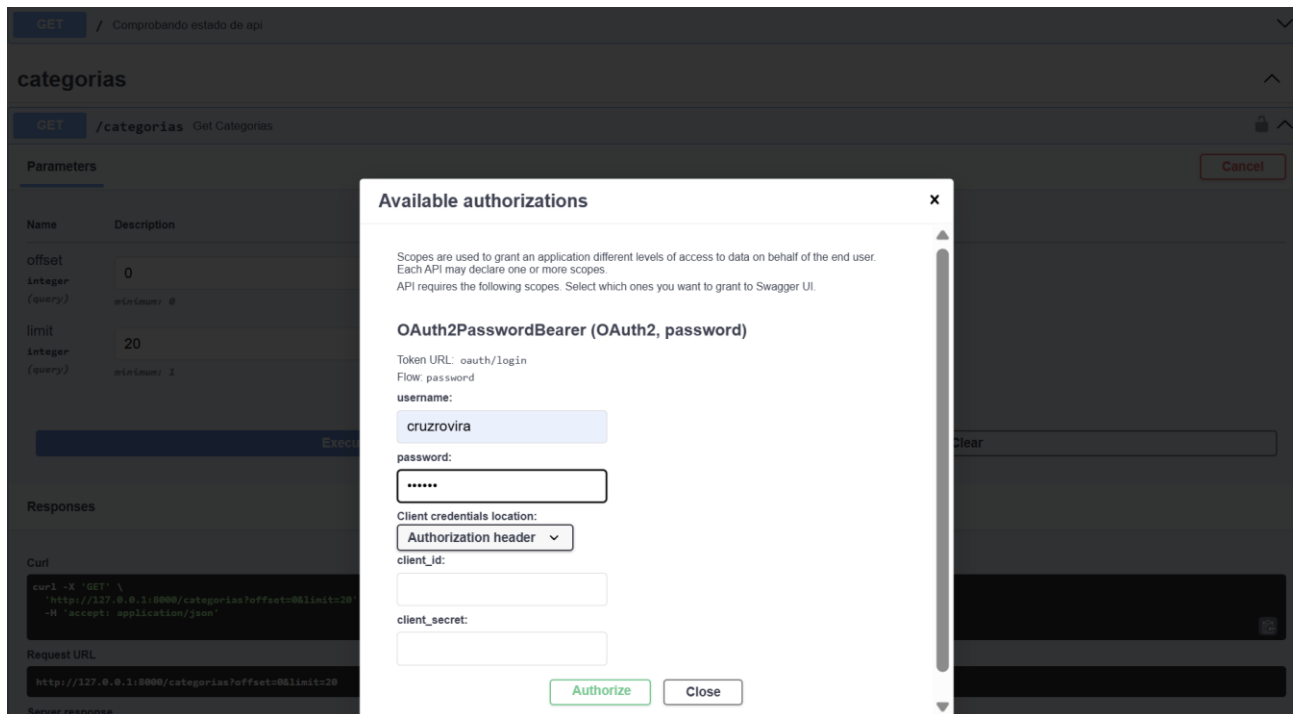
16. Que pasa si utilizamos la documentación para poder ver las categorías, denota que hoy aparecerá un candado esto indica que necesita realizar el login



17. Si realiza la petición sin realizar el login obtendrá una respuesta como la siguiente, indicando que no está autenticado



18. Si presiona sobre el candado abrirá un formulario para realizar el login



19. Luego de ello ya podra realizar la consulta como lo hacia antes

Proreccion de url por roles

Para poder proteger una url dado el rol de un usuario , es necesario primero proteger por token , una vez realizado eso recuerde que en el token hemos almacenado la informacion del usuario , id , username y id_rol , nos aprovecharemos de ello y agregaremos el codigo siguiente

```

1  from datetime import datetime
2  from fastapi import APIRouter, HTTPException, status, Query
3  from models.categoria import Categoria, CategoriaCreate, CategoriaUpdate
4  from sqlmodel import select
5  from config.session_Dependencia import SessionDeDependencia
6  from config.segurity_Dependencia import Token_Dependencia
7
8  router = APIRouter()
9
10
11 @router.get("/categorias", response_model=list[Categoria], status_code=status.HTTP_200_OK)
12 async def get_categorias(session: SessionDeDependencia,
13                          token: Token_Dependencia,
14                          offset: int = Query(0, ge=0),
15                          limit: int = Query(20, ge=1)):
16
17     if token['id_rol'] != 1:
18         raise HTTPException(status_code=status.HTTP_403_FORBIDDEN,
19                             detail="No tienes permisos para acceder a esta información")
20
21     # Obtener los datos del token decodificado
22     consulta = select(Categoria).offset(offset).limit(
23         limit) # Limitar a 'limit' resultados por defecto
24     resultado_de_consulta = session.exec(consulta)
25     return resultado_de_consulta.all()
26     # return session.exec(select(Categoria)).all()

```



EJERCICIOS DE PRÁCTICA:

- Realice los cambios correspondientes para poder agregar login a su api
- Y realizar la proyección de las ur con tokens

Tabla / recurso	Acción	ADMIN	VENDEDOR	Restricción recomendada
ROLES	Crear	✓	✗	Solo administrador
	Leer	✓	✗	Vendedor no necesita administrar roles
	Actualizar	✓	✗	Solo administrador
	Eliminar	✓	✗	No eliminar roles que tengan usuarios asociados
USUARIOS	Crear	✓	✗	Solo administrador crea vendedores/administradores
	Leer	✓	⚠	Vendedor solo puede consultar su propio usuario
	Actualizar	✓	⚠	Vendedor solo sus datos personales; no puede cambiar su rol
	Eliminar	✓	✗	Solo administrador
CATEGORÍAS	Crear	✓	✗	Administración del catálogo
	Leer	✓	✓	Necesario para consultar productos
	Actualizar	✓	✗	Solo administrador
	Eliminar	✓	✗	Validar que no tenga productos asociados
PRODUCTOS	Crear	✓	✗	Solo administrador
	Leer	✓	✓	Vendedor necesita consultar precios y existencia
	Actualizar	✓	✗	Evita que vendedor cambie precios o stock manualmente
	Eliminar	✓	✗	Solo administrador
CLIENTES	Crear	✓	✓	Vendedor registra clientes durante una venta
	Leer	✓	✓	Necesario para seleccionar clientes

	Actualizar	✓	✓	Puede corregir información del cliente
	Eliminar	✓	✗	Evitar pérdida de relaciones con ventas
VENTAS	Crear	✓	✓	id_usuario debe obtenerse del usuario autenticado
	Leer	✓	✓	Admin ve todas; vendedor solamente sus ventas
	Actualizar	✓	✓	Vendedor solo sus ventas y mientras estén editables
	Eliminar	⚠	✗	Mejor anular la venta que eliminarla físicamente
TIPO_PAGO	Crear	✓	✗	Catálogo administrado por ADMIN
	Leer	✓	✓	Vendedor necesita seleccionarlo al vender
	Actualizar	✓	✗	Solo administrador
	Eliminar	✓	✗	Validar que no esté siendo utilizado
DETALLE_VENTAS	Crear	✓	✓	Solo sobre ventas del vendedor
	Leer	✓	✓	Solo detalles de sus propias ventas
	Actualizar	✓	⚠	Solo mientras la venta sea editable
	Eliminar	✓	⚠	Solo mientras la venta no esté finalizada